

# Dzień 3

## Scheduling, autoscaling, deployment

## Plan dnia

1. **Init containers** — pre-start setup
2. **QoS, ResourceQuota, LimitRange**
3. **Metrics Server + HPA**
4. **Scheduling**: affinity, taints, topology spread
5. **Pod Priority + Preemption**
6. **Network Policies**
7. **Node maintenance** (drain, PDB)
8. **Strategie wdrożeń** (Canary deep dive)

→ Repo: `day3/`

## Init Containers (D3/01)

Uruchamiają się **sekwencyjnie przed** main containers.

### Use cases:

- Czekaj na Service / DB
- Schema migration
- Fetch secrets / config
- chown / chmod volumes

**Native sidecars (K8s 1.29+):** `initContainers + restartPolicy: Always` = sidecar startujący przed main

→ `day3/01_init_containers/`

## QoS Classes (D3/02)

| Klasa      | Warunek                           | OOM-kill priority |
|------------|-----------------------------------|-------------------|
| Guaranteed | <code>requests == limits</code>   | killed last       |
| Burstable  | <code>requests &lt; limits</code> | medium            |
| BestEffort | brak                              | killed first      |

### Produkcja:

- Guaranteed: critical (DB, payment)
- Burstable: typowy web
- BestEffort: batch / dev

→ `day3/02_QoS/01_pod_limits/`

## ResourceQuota + LimitRange

```
apiVersion: v1
kind: ResourceQuota
spec:
  hard:
    requests.cpu: "10"
    requests.memory: 20Gi
    nvidia.com/gpu: "4"
    pods: "50"
---
kind: LimitRange
spec:
  limits:
    - type: Container
      default: { cpu: 200m, memory: 256Mi }
      defaultRequest: { cpu: 100m, memory: 128Mi }
```

→ day3/02\_QoS/02\_limitrange , 03\_resource\_quota

## Metrics Server (D3/03)

Wymóg dla `kubectl top`, HPA, VPA.

```
kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/...  
# K3d/Kind: dodaj --kubelet-insecure-tls
```

```
kubectl top nodes  
kubectl top pods
```

⚠ Point-in-time, brak historii — Prometheus do długoterminowej (D5/04)

→ `day3/03_metrics_server/`

## HPA — Horizontal Pod Autoscaler (D3/04)

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
spec:
  scaleTargetRef: { name: my-app }
  minReplicas: 2
  maxReplicas: 50
  metrics:
    - type: Resource
      resource:
        name: cpu
        target: { type: Utilization, averageUtilization: 70 }
```

- **Custom metrics** przez Prometheus Adapter
- **VPA** = vertical (zmienia `requests/limits` )
- **Cluster Autoscaler / Karpenter** = dodaje nody (D5/07)

→ `day3/04_HPA/`

## Scheduling — 3 mechanizmy (D3/06)

| Mechanizm            | Cel                          |
|----------------------|------------------------------|
| Node/Pod Affinity    | preferuj/wymagaj lokalizacji |
| Taints + Tolerations | "nie planuj tu bez zgody"    |
| Topology Spread      | rozrzuć po domenach          |

**Komplementarne, nie alternatywne.**

Produkcja: GPU pool ( `taint` + `nodeSelector` ), spot instances ( `taint` ), multi-zone HA (TSC)

→ `day3/06_scheduling_rules/`



## PriorityClass + Preemption (D3/07)

```
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata: { name: high-priority }
value: 1000
preemptionPolicy: PreemptLowerPriority
```

- Wyższy priority **wywłaszcza** niższe gdy klaster pełny
  - `preemptionPolicy: Never` — wysoki priority bez preemption
  - Built-in: `system-cluster-critical`, `system-node-critical`
- `day3/07_pod_priority/`

## NetworkPolicy (D3/08)

**Default:** wszystkie Pody mogą rozmawiać z wszystkimi.

**Default-deny** + selective allow:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
spec:
  podSelector: { matchLabels: { app: backend } }
  policyTypes: [Ingress, Egress]
  ingress:
    - from:
      - podSelector: { matchLabels: { app: frontend } }
```

⚠ **Wymaga CNI z policy support:** Calico, Cilium (NIE Flannel)

→ day3/08\_network\_policy/

## Node maintenance (D3/09)

```
kubectl cordon $NODE      # nie planuj nowych
kubectl drain $NODE \
  --ignore-daemonsets \
  --delete-emptydir-data
# patch / reboot
kubectl uncordon $NODE
```

**PodDisruptionBudget** — drain respektuje:

```
spec:
  minAvailable: 75%      # lub maxUnavailable: 1
  selector: { matchLabels: { app: critical } }
```

→ day3/09\_node\_maintenance/

## Strategie wdrożeń — 6 wariantów (D3/05)

|               | Downtime | Resource | Rollback        |
|---------------|----------|----------|-----------------|
| Recreate      | TAK      | 1×       | trudny          |
| Rolling       | NIE      | 1.25×    | wolny           |
| Blue/Green    | NIE      | 2×       | natychmiastowy  |
| <b>Canary</b> | NIE      | 1.x×     | natychmiastowy  |
| A/B           | NIE      | 2×       | per cohort      |
| Shadow        | NIE      | 2×       | n/a (read-only) |

**Argo Rollouts** = automatyzacja Canary z metryk Prometheus

→ `day3/05_Canary/strategies.md`

## Canary — przykład Argo Rollouts

```
apiVersion: argoproj.io/v1alpha1
kind: Rollout
spec:
  strategy:
    canary:
      steps:
        - setWeight: 20
        - pause: { duration: 5m }
        - analysis: # auto-rollback jeśli error rate > 1%
            templates: [{ templateName: success-rate }]
        - setWeight: 50
        - pause: { duration: 10m }
        - setWeight: 100
```

→ `day3/05_Canary/canary-demo/`

## Podsumowanie D3

- ✓ Init containers + QoS + ResourceQuota
- ✓ Metrics Server + HPA
- ✓ Affinity, taints, TSC
- ✓ PriorityClass + preemption
- ✓ NetworkPolicy + drain z PDB
- ✓ Strategie wdrożeń (porównanie 6)

**Jutro:** Security deep dive — PSA, RBAC zaawansowany, Vault, kube-bench, Trivy, Falco, OPA